> aula_21.10

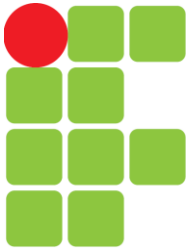
JavaScript

Parte 4 · JS na página

Desenvolvimento de Sistemas Web · 21/10

Prof. Thiago Araújo

IFPE Campus Palmares · ADS 3º período



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

> mapa_da_aula

Mapa da aula

01

{ }

Objetos e JSON

02

</>

O DOM

03

map

Listas a partir de dados

04

on

Eventos e delegação

05

form

Formulários acessíveis

06

10

Projeto valendo nota

> pergunta?

Como o cardápio do iFood sai do servidor e vira botão na sua tela?

01

Objetos e JSON

Representar, guardar e trocar dados



Objeto agrupa os **dados** de uma mesma coisa, cada um com um nome.

Chave: valor, com vírgula entre os pares.

```
const item = {
  id: 1,
  nome: 'Coxinha',
  preco: 5,
  categoria: 'salgado',
}

item.nome     // 'Coxinha'
item.preco    // 5
```

> analogia

Classe é a forma, objeto é o bolo

No código

```
class Item {
  constructor(nome, preco) {
    this.nome = nome
    this.preco = preco
  }
  subtotal(qtd = 1) {
    return this.preco * qtd
  }
}
const c = new Item('Coxinha', 5)
c.subtotal(3) // 15
```

≈

No dia a dia

new

A **classe** é a forma de bolo.

Cada **new** assa um bolo novo, com o seu sabor.

> erro_comum

Objeto dentro de texto

✗ Evite

```
const cliente = {
  nome: 'Ana',
  zap: '81 99999-0000',
}
`Olá, ${cliente}!`
// 'Olá, [object Object]!'
```

O JS não sabe **qual** campo mostrar.

✓ Prefira

```
`Olá, ${cliente.nome}!`
// 'Olá, Ana!'

JSON.stringify(cliente)
// '{"nome":"Ana",...}'
```

Escolha o campo, ou use **JSON**.

JSON é **texto** com cara de objeto: é assim que os dados viajam.

```
const item = { nome: 'Coxinha' }

const texto =
  JSON.stringify(item)
// '{"nome":"Coxinha"}'

const volta =
  JSON.parse(texto)
// { nome: 'Coxinha' }
```

`stringify` arruma a mala; `parse` desfaz. Texto quebrado? `try/catch`.



> erro_comum

Salvar no localStorage

✗ Evite

```
const carrinho = [
  { nome: 'Coxinha', preco: 5 }]
localStorage.setItem(
  'carrinho', carrinho)
// gravou '[object Object]'
```

Sem JSON, os itens **se perdem**.

✓ Prefira

```
localStorage.setItem(
  'carrinho',
  JSON.stringify(carrinho))
// '[{"nome":"Coxinha",...}]'
```

`stringify` vira texto **antes** de salvar.

Para ler, faça o caminho de volta: **texto → objeto**.

O `??` (aula 1) entra só quando não veio nada salvo.

```
const salvo =
  localStorage.getItem('carrinho')
// 1ª visita: null

const carrinho =
  JSON.parse(salvo) ?? []
// null ?? [] → vazio
```




> checkpoint

O que aparece no console?

```
localStorage.setItem('n', 10)
typeof localStorage.getItem('n')
```

A `'number'`

B `'string'`

C `'object'`

D `'undefined'`

> resposta

O que aparece no console?

```
localStorage.setItem('n', 10)
typeof localStorage.getItem('n')
```

A `'number'`

B `'string'`

C `'object'`

D `'undefined'`

02

O DOM

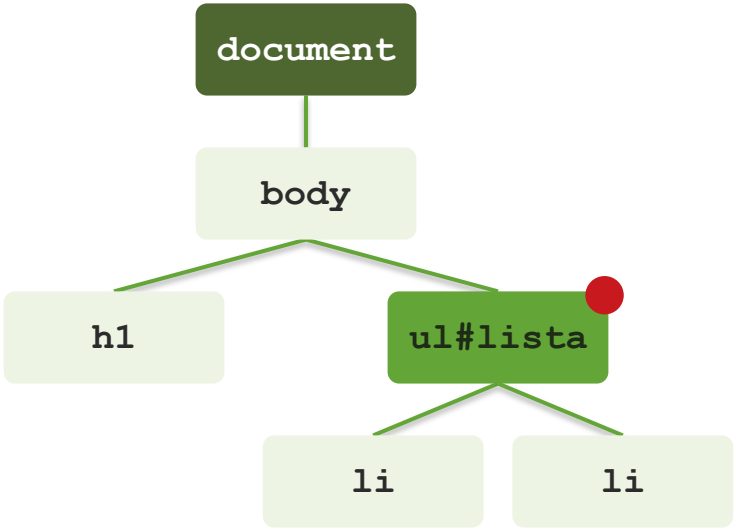
A página como uma árvore de objetos



> conceito

O HTML vira uma árvore

```
<body>
  <h1>Cardápio</h1>
  <ul id="lista">
    <li>Coxinha</li>
    <li>Pastel</li>
  </ul>
</body>
```



Cada tag vira um **objeto** que o JavaScript pode ler e mudar.

> erro_comum

Onde colocar o script

✗ Evite

```
<head>
  <script src="app.js">
  </script>
</head>
<!-- roda cedo demais -->
```

Erro: **Cannot read properties of null**.

✓ Prefira

```
<head>
  <script src="app.js"
          defer></script>
</head>
<!-- espera o HTML -->
```

`defer`: roda depois de montar a página.

Selecionar e alterar

— **querySelector**
o primeiro que casa

txt — **textContent**
troca o texto, sem risco

class — **classList**
add, remove, toggle

data — **dataset**
guarda o id no elemento

```
const t = document
  .querySelector('#titulo')
t.textContent = 'Cardápio'
t.classList.add('destaque')

const botao = document
  .querySelector('button')
botao.dataset.id = '3'
// <button data-id="3">
```




> atenção!

innerHTML com dado de fora: XSS

✗ Evite

```
const campo = document
  .querySelector('#nome')
const saudacao = document
  .querySelector('#saudacao')
saudacao.innerHTML =
  `Olá, ${campo.value}!`
// valor: <img src=x onerror=...>
```

O navegador **executa** o que foi digitado.

✓ Prefira

```
const campo = document
  .querySelector('#nome')
const saudacao = document
  .querySelector('#saudacao')
saudacao.textContent =
  `Olá, ${campo.value}!`
// vira só texto
```

`textContent` nunca interpreta HTML.

> checkpoint

Qual é seguro para exibir o nome digitado pelo usuário?

A `innerHTML`

B `textContent`

C `outerHTML`

D `insertAdjacentHTML`

> resposta

Qual é seguro para exibir o nome digitado pelo usuário?

A `innerHTML`

B `textContent`

C `outerHTML`

D `insertAdjacentHTML`

03

Listas a partir de dados

A tela é o reflexo do array



Criar um elemento em 4 passos

1 — 2 — 3 — 4

Criar
um elemento novo, solto

Preencher
texto e classes

Inserir
no lugar certo da árvore

Remover
quando não serve mais

```
const lista = document.querySelector('ul')  // a lista da página
const li = document.createElement('li')     // 1
li.textContent = 'Pastel'                   // 2
lista.append(li)                            // 3
li.remove()                                 // 4
```




> na_prática

A função renderizar()

```
const cardapio = [{ id: 1, nome: 'Coxinha', preco: 5 }]
const lista = document.querySelector('ul')
function renderizar(itens) {
  lista.replaceChildren()
  for (const item of itens) {
    const li = document.createElement('li')
    li.textContent = `${item.nome}: R$ ${item.preco}`
    const btn = document.createElement('button')
    btn.textContent = 'Adicionar'
    btn.dataset.id = item.id
    li.append(btn)
    lista.append(li)
  }
}
renderizar(cardapio)
```

1. Os dados vivem **no array**.
2. **Limpa** antes de redesenhar.
3. O botão guarda o **id** no **`dataset`**.



Filtrar = montar **outra** lista e desenhar de novo.

O cardápio original **não muda**: ele continua com os 3 itens.

```
const cardapio = [
  { nome: 'Coxinha', preco: 5 },
  { nome: 'Açaí', preco: 18 },
  { nome: 'Suco', preco: 6 },
]

const baratos = []
for (const item of cardapio) {
  if (item.preco <= 6) {
    baratos.push(item)
  }
}
renderizar(baratos)
// renderizar(): slide anterior
```

> checkpoint

Onde o Suco aparece na página?

```
const ul = document.querySelector('ul')
const li = document.createElement('li')
li.textContent = 'Suco'
```

A No fim da lista

B No começo da lista

C Em lugar nenhum

D No lugar da `ul`

> resposta

Onde o Suco aparece na página?

```
const ul = document.querySelector('ul')
const li = document.createElement('li')
li.textContent = 'Suco'
```

A — No fim da lista

B — No começo da lista

C — **Em lugar nenhum**

D — No lugar da `ul`

04

Eventos e delegação

O usuário faz algo, uma função reage

{ } </> map on form 10

> analogia

Evento é campainha

No código

```
const botao = document
  .querySelector('button')

botao.addEventListener(
  'click',
  () => console.log('oi')
)
```

≈

No dia a dia

`on`

O **evento** é a campainha tocando.

O **listener** é quem atende a porta.

> conceito

A cada tecla, monte a lista dos **achados** e desenhe de novo.

É o laço do filtro, só que dentro de um evento.

```
const busca = document
  .querySelector('#busca')

function buscar() {
  const termo = busca.value
  const achados = []
  for (const item of cardapio) {
    if (item.nome.includes(termo)) {
      achados.push(item)
    }
  }
  renderizar(achados)
}

busca.addEventListener('input',
  buscar)
// cardapio e renderizar(): bloco 3
```




O botão diz **qual** item: procure no cardápio.

`dataset` devolve texto: use `Number()` antes do `===`.

```
const cardapio = [
  { id: 1, nome: 'Coxinha', preco: 5 },
  { id: 2, nome: 'Pastel', preco: 8 },
]
let carrinho = []

function adicionar(id) {
  for (const item of cardapio) {
    if (item.id === Number(id)) {
      carrinho.push(item)
    }
  }
  mostrarTotal()   // próximo slide
}
```




Um laço **soma** o total do carrinho.

O acumulador da aula de laços: começa em 0 e soma a cada volta.

```
const total = document
  .querySelector('#total')

function mostrarTotal() {
  let soma = 0
  for (const item of carrinho) {
    soma += item.preco
  }
  total.textContent = `R$ ${soma}`
}
// carrinho: slide anterior
```

Delegação de eventos

✗ Evite

```
const lista = document
  .querySelector('#cardapio')
const botoes = lista
  .querySelectorAll('button')
for (const b of botoes) {
  b.addEventListener('click',
    adicionar)
}
```

Botão criado depois fica **surdo**.

✓ Prefira

```
const lista = document
  .querySelector('#cardapio')
lista.addEventListener('click',
  (e) => {
    const id = e.target.dataset.id
    if (id) {
      adicionar(id)
    }
  })
```

Um listener no **pai** serve a todos.



> checkpoint

O que está errado?

```
const botao = document.querySelector('button')
function adicionar() { console.log('adicionou') }
botao.addEventListener('click', adicionar())
```

A Nada

B `adicionar()` roda na hora, não no clique

C Falta ponto e vírgula

D Deveria ser `'onclick'`

> resposta

O que está errado?

```
const botao = document.querySelector('button')
function adicionar() { console.log('adicionou') }
botao.addEventListener('click', adicionar())
```

A Nada

B **`adicionar()` roda na hora, não no clique**

C Falta ponto e vírgula

D Deveria ser `'onclick'`



05

Formulários acessíveis

submit, validação e o leitor de tela

{ } </> map on form 10

> erro_comum

O formulário que recarrega

✗ Evite

```
const form = document
  .querySelector('form')
form.addEventListener(
  'submit', () => {
    console.log('enviou')
  })
// a página recarrega!
```

Os dados **somem** com o recarregamento.

✓ Prefira

```
const form = document
  .querySelector('form')
form.addEventListener(
  'submit', (e) => {
    e.preventDefault()
    console.log('enviou')
  })
```

`preventDefault` segura o envio padrão.



Leia cada campo pelo `.value` e valide com `if`.

O HTML ajuda: **`required`** e **`type`** validam o básico.

```
const form = document.querySelector('#pedido')
const inputNome = document.querySelector('#nome')
const inputZap = document.querySelector('#zap')
const aviso = document.querySelector('#aviso')
const msg = document.querySelector('#mensagem')

form.addEventListener('submit', (e) => {
  e.preventDefault()
  const nome = inputNome.value
  const zap = inputZap.value
  if (nome === '' || zap === '') {
    aviso.textContent = 'Preencha!'
    return
  }
  msg.textContent = `Pedido: ${nome}`
})
```

Se a tela muda, **anuncie**: uma região viva lê as mudanças.

A região existe **vazia** desde o início; o JS só troca o texto.

```
<p id="aviso" role="status"
   aria-live="polite"></p>

const aviso = document
  .querySelector('#aviso')
aviso.textContent =
  'Coxinha adicionada.'
// o leitor de tela anuncia
```




> na_prática

Erro que o leitor de tela lê

lanchonete-do-campus/pedido

WhatsApp

81 9

Informe o número com DDD.

Fazer pedido

O leitor lê o **erro** com o campo.

O foco vai ao primeiro erro.

```
const campo = document
  .querySelector('#zap')
const erro = document
  .querySelector('#zap-erro')
campo.setAttribute(
  'aria-invalid', 'true')
erro.textContent =
  'Informe o número com DDD.'
campo.focus()
```

> checkpoint

Sem `e.preventDefault()` no submit, o que acontece?

A Nada de diferente

B A página recarrega e os dados somem

C Um erro no console

D O formulário não envia



> resposta

Sem `e.preventDefault()` no submit, o que acontece?

A. Nada de diferente

B. A página recarrega e os dados somem

C. Um erro no console

D. O formulário não envia

06

Projeto valendo nota

Cardápio Digital da lanchonete

{ } </> map on form 10

> na_prática

Projeto: Cardápio Digital



Um **cardápio digital** com busca, carrinho e pedido.

Usa **tudo** do JS: dados, DOM, eventos e formulário.

Rubrica: 10 pontos

Dados	**`JSON.parse`** do cardápio, com 10 itens	1,0
Renderizar	lista com **`createElement`** e **`textContent`**	2,0
Busca	filtra pelo nome a cada tecla	1,5
Carrinho	adicionar e remover com **delegação**; total somado	2,0
Salvar	o carrinho volta depois do F5	1,0
Pedido	formulário validado e mensagem do pedido	1,5
Acessível	labels, botões reais e **`role="status"`**	1,0

Bônus até +1 · **innerHTML com dado do usuário: −1**



Roteiro sugerido



Dados

JSON e `renderizar`

Busca

evento input e laço

Carrinho

delegação e soma

Salvar

localStorage com JSON

Pedido

formulário e mensagem

Regras do projeto

2x — **Em duplas**
os dois explicam qualquer parte do código

dia — **Prazo: 04/11, 23h59**
pelo Classroom, em .zip com os 3 arquivos

IA — **IA pode ajudar a entender**
mas a dupla defende cada linha

? — **Defesa de 3 minutos**
eu escolho um trecho e vocês explicam



> desafio

Comece agora: etapa 1

⏱ 40 min

1. Baixe o projeto inicial no Classroom.
2. Converta o texto com **`JSON.parse`**.
3. Escreva **`renderizar(itens)`**.
4. Cada item com nome, preço e botão.

Ao fim da aula: a lista na tela.

```
// app.js (projeto inicial)
// texto: o JSON que já vem
// no começo do arquivo
const cardapio =
  JSON.parse(texto)
const lista = document
  .querySelector('#cardapio')

function renderizar(itens) {
  // sua vez!
}
renderizar(cardapio)
```

> resumo

Leve para casa

{ }

Dados viram **objetos**; o JSON leva e traz.

</>

A tela é o **reflexo** dos dados.

on

Eventos e **delegação** dão vida à página.

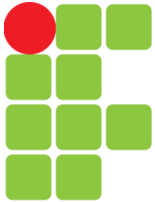
> próxima_aula

Assíncrono: fetch e APIs

próxima aula · traga o projeto rodando

Projeto: entrega até 04/11, 23h59 no Classroom
Dúvidas do projeto no começo da próxima aula

Prof. Thiago Araújo · IFPE Campus Palmares · ADS 3º período



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO